

1. Layer wise speedups . CPU unless specified.
2. Three step scheme
 - a. Train with group regularization
 - b. Prune the groups with lowest magnitude
 - c. Finetune
- 3.

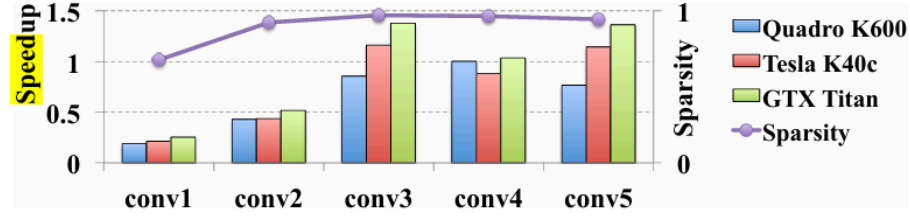


Figure 1: Evaluation speedups of AlexNet on GPU platforms and the sparsity. conv1 refers to convolutional layer 1, and so forth. Baseline is profiled by GEMM of cuBLAS. The sparse matrixes are stored in the format of Compressed Sparse Row (CSR) and accelerated by cuSPARSE.

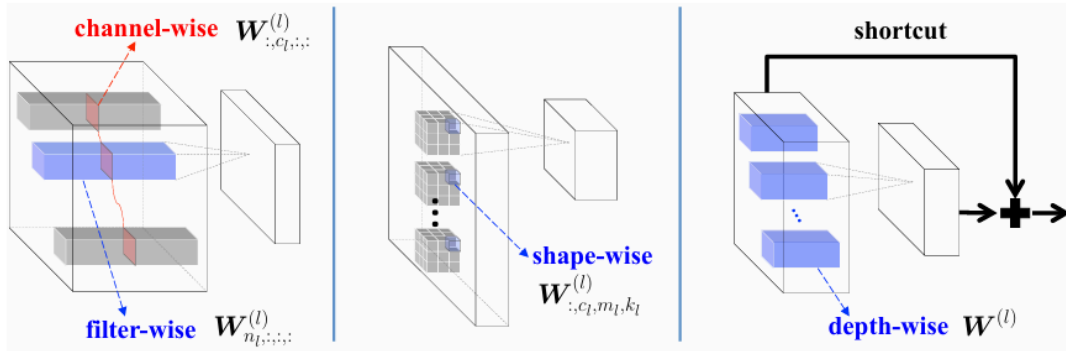


Figure 2: The proposed structured sparsity learning (SSL) for DNNs. Weights in filters are split into multiple groups. Through group Lasso regularization, a more compact DNN is obtained by removing some groups. The figure illustrates the filter-wise, channel-wise, shape-wise, and depth-wise structured sparsity that were explored in the work.

We evaluated the effectiveness of our SSL using published models on three databases – MNIST, CIFAR-10, and ImageNet. Without explicit explanation, SSL starts with the network whose weights are initialized by the baseline, and speedups are measured in matrix-matrix multiplication by Caffe in a single-thread Intel Xeon E5-2630 CPU .

Table 1: Results after penalizing unimportant filters and channels in *LeNet*

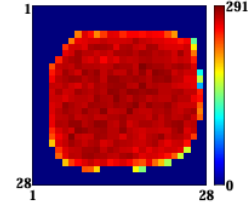
<i>LeNet</i> #	Error	Filter # [§]	Channel # [§]	FLOP [§]	Speedup [§]
1 (<i>baseline</i>)	0.9%	20—50	1—20	100%—100%	1.00×—1.00×
2	0.8%	5—19	1—4	25%—7.6%	1.64×—5.23×
3	1.0%	3—12	1—3	15%—3.6%	1.99×—7.44×

[§]In the order of *conv1*—*conv2*Table 2: Results after learning filter shapes in *LeNet*

<i>LeNet</i> #	Error	Filter size [§]	Channel #	FLOP	Speedup
1 (<i>baseline</i>)	0.9%	25—500	1—20	100%—100%	1.00×—1.00×
4	0.8%	21—41	1—2	8.4%—8.2%	2.33×—6.93×
5	1.0%	7—14	1—1	1.4%—2.8%	5.19×—10.82×

[§] The sizes of filters after removing zero shape fibers, in the order of *conv1*—*conv2*

<i>MLP</i> #	Error	Neuron # per layer [§]	FLOP per layer [§]
1 (<i>baseline</i>)	1.43%	784—500—300—10	100%—100%—100%
2	1.34%	469—294—166—10	35.18%—32.54%—55.33%
3	1.53%	434—174—78—10	19.26%—9.05%—26.00%

[§]In the order of *input layer*—*hidden layer 1*—*hidden layer 2*—*output layer*

(a)

(b)

Figure 4: (a) Results of learning the number of neurons in *MLP*. (b) the connection numbers of input neurons (*i.e.* pixels) in *MLP* 2 after SSL.Table 3: Learning row-wise and column-wise sparsity of *ConvNet* on CIFAR-10

<i>ConvNet</i> #	Error	Row sparsity [§]	Column sparsity [§]	Speedup [§]
1 (<i>baseline</i>)	17.9%	12.5%—0%—0%	0%—0%—0%	1.00×—1.00×—1.00×
2	17.9%	50.0%—28.1%—1.6%	0%—59.3%—35.1%	1.43×—3.05×—1.57×
3	16.9%	31.3%—0%—1.6%	0%—42.8%—9.8%	1.25×—2.01×—1.18×

[§]in the order of *conv1*—*conv2*—*conv3*

Table 4: Sparsity and speedup of *AlexNet* on ILSVRC 2012

#	Method	Top1 err.	Statistics	conv1	conv2	conv3	conv4	conv5
1	ℓ_1	44.67%	sparsity	67.6%	92.4%	97.2%	96.6%	94.3%
			CPU $\times$	0.80	2.91	4.84	3.83	2.76
			GPU $\times$	0.25	0.52	1.38	1.04	1.36
2	SSL	44.66%	column sparsity	0.0%	63.2%	76.9%	84.7%	80.7%
			row sparsity	9.4%	12.9%	40.6%	46.9%	0.0%
			CPU $\times$	1.05	3.37	6.27	9.73	4.93
			GPU $\times$	1.00	2.37	4.94	4.03	3.05
3	pruning [7]	42.80%	sparsity	16.0%	62.0%	65.0%	63.0%	63.0%
4	ℓ_1	42.51%	sparsity	14.7%	76.2%	85.3%	81.5%	76.3%
			CPU $\times$	0.34	0.99	1.30	1.10	0.93
			GPU $\times$	0.08	0.17	0.42	0.30	0.32
5	SSL	42.53%	column sparsity	0.00%	20.9%	39.7%	39.7%	24.6%
			CPU $\times$	1.00	1.27	1.64	1.68	1.32
			GPU $\times$	1.00	1.25	1.63	1.72	1.36